ProbeFlow 장문 한국어 해설서

공유 입력·내부 캐시·보완 정보가 선택적 부품 상태 예측을 어떻게 바꾸는가 작성일: 2026-10-12.
첫 주 연구 설계 원고와 함께 읽는 설명서.

이 연구의 중심 질문은 예측기를 고정한 상태에서 어떤 분석을 어떤 순서로 요청하고 언제 답하거나 보류할 것인가이다. 서로 다른 분석이 같은 입력을 쓰고, 서버가 계산한 값 중 일부만 정책에 공개되며, 하나씩 보면 쓸모없던 정보가 함께 관측되면 결정적일 수 있다는 세 조건을 함께 다룬다. 새로운 대형 예측 모델을 개발하거나 실제 설비를 제어하는 연구로 해석하지 않는다.

4페이지 영문 원고 (paper.json), 방법 계약 (METHODS_CONTRACT.json), 근거·주장 연결 (SOURCES.json)이 이 설명서와 같은 정의를 사용한다. 해설서의 계산 예시는 명시한 가상 세계의 논리 확인이며 실제 산업 성능표가 아니다. 검증된 소프트웨어 동작, 실제 파일 준비, 원문 저자의 보고, 설계 제안과 미검증 가설을 구분한다.

1. 왜 분석의 순서가 연구 문제가 되는가

유압 장치의 한 운전 주기를 기록했다고 하자. 분석 A는 압력 평균을 반환하고, 분석 B는 압력 변화량을 반환하며, 분석 C는 압력과 유량을 함께 읽어 숫자 요약을 만든다. A와 B는 같은 압력 창을 사용할 수 있다. A를 실행한 뒤 B를 실행할 때 압력 입력을 이미 읽었다면 두 번째 조회에 전체 읽기 비용을 다시 붙이는 것은 실제 실행 구조와 맞지 않는다. 반대로 C가 내부에서 압력 평균과 유량 평균을 계산했다고 해서 정책이 두 평균을 모두 본 것은 아니다. C의 계약이 조합한 숫자 하나를 반환한다면 정책은 그 숫자만 얻는다.

이때 "현재 어떤 feature를 봤는가"와 "서버가 어떤 입력을 이미 확보했는가"라는 질문이 갈라진다. 첫 번째는 예측이 사용할 정보를 결정하고, 두 번째는 다음 행동의 한계비용을 결정한다. 정책은 같은 확신도를 갖고도 비용 상태 때문에 다른 행동을 선택할 수 있다. 좋은 예측값을 많이 얻는 것이 목적이라도 비용과 정보 접근을 혼동하면 방법의 우위를 잘못 측정한다.

또 하나의 문제는 보완 정보다. 저렴한 측정 하나가 그 자체로는 라벨을 잘 설명하지 못하지만 다음에 어떤 측정을 해야 할지 알려줄 수 있다. 또는 두 측정을 함께 보아야 상태가 구별될 수 있다. 한 단계 뒤의 예측 오차 감소만 계산하는 정책은 첫 측정을 선택하지 않을 수 있다. 이러한 지연된 가치 자체는 이미 active feature acquisition 문헌의 핵심 주제다. ProbeFlow가 새로 발견했다고 주장할 부분은 아니다.

우리의 좁은 질문은 공유 source와 공개 경계를 갖는 분석 인터페이스에서 어떤 상태 표현과 비용 회계가 공정한 비교를 가능하게 하고, 짧은 continuation을 고려하는 정책이 실제로 추가 가치를 주는가이다. 여기서 continuation은 현재 행동 뒤에 가능한 다음 행동들의 연속을 뜻한다. 첫 행동의 즉시 효과와 이후 선택권의 가치를 구별해서 평가한다.

예를 들어 모든 센서 읽기가 거의 무료이고 mean 계산도 매우 싸다면, 복잡한 planner보다 모든 분석을 한 번 계산하는 규칙이 나을 수 있다. 반대로 압력 읽기에 큰 비용이 들고 그 입력으로 여러 분석을 만들 수 있다면 첫 읽기 뒤의 재사용이 중요해진다. 어느 경우에 해당하는지 실제 조회·계산·정책 실행 비용을 측정해야 하며, "공유 입력이 있으니 새 방법이 유리하다"는 결론은 자동으로 나오지 않는다.

가설을 검증할 때 정책끼리 같은 예측기와 같은 도구를 사용하도록 고정하는 이유도 여기에 있다. 새 정책만 더 강한 예측기를 사용하면 개선의 원인이 분석 순서인지 모델 용량인지 알 수 없다. 새 정책만 내부 prerequisite를 무료로 받아도 개선의 원인이 계획인지 정보 누출인지 알 수 없다. 과학적 실험은 실제로 바꾼 변수를 설명할 수 있어야 한다.

ProbeFlow의 첫 주 목표는 모든 산업 조건에서 우월한 정책을 선언하는 것이 아니다. 계산 가능한 반례로 가정의 경계를 드러내고, 그 경계를 실제 기록 실험에서 검사할 수 있는 계약을 만드는 것이다. 기존 greedy나 pair lookahead가 충분하면 그 결과를 그대로 보고한다. 비용을 포함한 효과가 없다는 결과도 설계의 유효한 결과다.

2. 지금 있는 구현과 아직 없는 산업 결과

현재 근거는 세 층으로 나뉜다. 첫째는 v3에서 새로 작성한 표준 라이브러리 참조 kernel이다. 둘째는 v4에서 별도로 확보한 실제 Hydraulic 입력과 형식 감사다. 셋째는 이 v5에서 작성한 논문·해설·방법 계약이다. 이 세 층의 상태를 하나의 "실험 완료"로 합치지 않는다.

v3 METHODS (../../skill_research_v3_20261012/probeflow/METHODS.md)와 기존 검증 영수증 (../../skill_research_v3_20261012/probeflow/validation/final_checks_20261012.json)은 typed dependency

DAG, source identity, 원본 캐시와 공개 mask의 분리, 실패 비용 유지, greedy와 작은 exact reference를 설명한다. 그 영수증에는 25개 소프트웨어 검사가 통과했다고 기록되어 있다. 이 작성 작업에서 그 25개를 다시 실행한 것으로 표시하지 않는다. 현재 코드를 읽어 설명과 구현의 범위를 대조했다.

참조 세계는 네 binary 값을 조합한 16개 authored world다. 같은 생성 설정 하나에서 직접 만든 세계이며, 학습 데이터가 아니다. 예측기는 그 세계에 균등 prior를 둔 뒤 실제 공개된 분석 결과와 일치하는 세계에 조건부 확률을 준다. 숨겨진 현재 episode의 정답을 정책에 전달해서 예측하는 구조와 다르지만, 데이터 생성과 예측 모델이 같은 정의를 공유한다는 한계는 남는다. 이 결과로 새로운 산업 상황에서의 예측 성능을 주장할 수 없다.

16개 episode에 세 availability mask, 네 예산, 여섯 정책을 적용해 1,152개 terminal trial이 생긴다. 1,152는 독립 관측 수가 아니다. 서로 다른 정책이 같은 episode를 본 횟수, 같은 episode에서 센서를 숨긴 경우, 예산을 바꾼 경우를 합친 수다. 고유 authored episode는 16개이고 생성 family는 하나이며 독립 과학 평가 집단은 0개다. 이 숫자를 산업 표본 수처럼 사용하지 않는다.

v4 확보 영수증 (../../skill_research_dataset_v4_20261012/data/hydraulic_selected/download_receipt.json)은 공식 Hydraulic ZIP의 바이트 수와 해시를 보존한다. ZIP은 76,601,704 bytes이고 SHA-256은 24128aad2ee45eea7e6b63ebbd9992cdf25d0483a2cebefbfc13bc69079af1f2다. 압력 PS1, 유량 FS1, 온도 TS1, profile을 제한적으로 추출했다. readiness audit (../../skill_research_dataset_v4_20261012/probeflow_readiness/runs/uci_selected_20261012/data_audit.json)는 이 파일들의 2,205개 주기와 열 수를 검사했다.

실제 원본은 확보 디렉터리에 보존되어 있지만 readiness 실행 자체의 snapshot은 profile만 담는다. 따라서 readiness package 안의 기록만으로 센서 전체를 재생할 수 있다고 말하면 틀리다. 센서는 별도 확보 원본과 파일별 해시를 연결해야 한다. profile 재해석으로 class·행 번호·안정성 요약을 확인하는 범위와 실제 센서 원본을 다시 읽는 범위를 구분한다.

현재 센서는 14개 물리 채널 중 3개만 준비되었다. 11개 채널이 없는 상태를 "센서 세 개면 네 부품을 충분히 진단한다"는 결과로 바꾸지 않는다. full-bank 실험을 선택하면 나머지 준비가 필요하고, restricted-bank 실험을 선택하면 세 채널로 제한한 실험이라고 명시해야 한다. 두 선택은 서로 다른 관측 환경이다.

정규화, 실제 categorical predictor 학습, acquisition policy 학습, calibration fitting, 최종 진단 benchmark는 수행되지 않았다. 독립 실험 블록을 보여주는 자료도 확보하지 못했다. 따라서 이 설명서가 제시하는 학습 절차는 앞으로 구현하고 잠가야 할 설계이고, 모든 산업 성능 결과는 NR이다. NR은 실패를 숨기는 표시가 아니라 아직 보고할 실측 결과가 없다는 상태다.

논문을 읽을 때는 문장의 주어를 확인하는 것이 도움이 된다. "kernel이 캐시를 분리한다"는 구현 문장이다. "파일이 2,205행이었다"는 준비 실행 문장이다. "정책이 더 좋은 terminal utility를 줄 것이다"는 가설 문장이다. 이 세 문장이 서로의 근거를 대신하지 않도록 SOURCES의 claim 연결을 유지한다.

3. 정답·예측·보류를 정확히 정의한다

주 평가 후보는 UCI Hydraulic 공식 데이터 (https://archive.ics.uci.edu/dataset/447/condition+monitoring+of+hydraulic+systems)다. 공식 profile은 냉각기, 밸브, 펌프 내부 누설, 축압기의 네 상태를 주기별로 제공한다. 연구에서는 네 범주형 head로 유지한다. 숫자 코드에 물리적인 의미가 있어도 첫 평가의 채점은 클래스 일치 여부로 고정한다.

head	허용 class	첫 평가에서의 채점
냉각기	3, 20, 100	맞으면 0, 틀리면 1
밸브	73, 80, 90, 100	맞으면 0, 틀리면 1
펌프 내부 누설	0, 1, 2	맞으면 0, 틀리면 1
축압기	90, 100, 115, 130	맞으면 0, 틀리면 1

3/4/3/4는 각 head의 클래스 수다. 네 값의 모든 조합이 같은 횟수로 관측되었다는 뜻은 아니다. 네 숫자를 한 정수로 합쳐 144-way 분류로 만드는 것도 첫 계약과 다르다. 3×4×3×4=144는 가능한 Cartesian 조합 수일 뿐 실제로 존재하는 조건 조합의 수는 원본에서 따로 확인해야 한다.

실제 정답 y와 예측 $\hat{y}$가 있을 때 component loss는 네 오분류 indicator의 평균이다.

$$\mathrm{ell}(\hat{y},y) = [1(\hat{y}1\neq y1)+1(\hat{y}2\neq y2)+1(\hat{y}3\neq y3)+1(\hat{y}4\neq y4)] / 4$$

예를 들어 정답이 (100,90,1,115), 예측이 (100,80,1,130)이면 두 head가 틀려 loss는 2/4=0.5다. exact-vector error는 네 값 중 하나라도 틀렸으면 1이므로 이 예에서는 1이다. 둘은 다른 질문을 답한다. component loss는 얼마나 많은 부품 상태를 틀렸는지, vector error는 전체 벡터를 완전히 맞췄는지 측정한다.

숫자 간 거리로 채점하면 또 다른 연구가 된다. 밸브 100을 90으로 틀린 것과 73으로 틀린 것을 다르게 벌주거나, 펌프 심각 누설을 정상으로 예측한 경우를 더 위험하게 볼 수 있다. 그러한 가중치가 필요할 수 있지만 실제 정비 위험 비용을 확보한 상태는 아니다. 첫 결과를 본 뒤 가중치를 바꿔 우위를 만들어서는 안 된다. 별도 가중 loss를 쓸 경우 의미와 값을 사전에 고정하고 단순 0/1 평균도 함께 보고한다.

정책은 diagnose와 abstain 중 하나로 종료한다. diagnose는 네 예측 코드를 확정한다. abstain은 답을 보류하며 λA라는 terminal penalty를 받는다. λA는 "보류하면 정말 이만큼의 손해가 난다"는 실측값이 아니라 연구 목적함수의 사전 선택이다. 실제 산업 λA는 미정이다. 가상 예시에서 0.3 또는 0.1을 사용한다고 해서 이 값이 유압 장치의 비용으로 검증된 것은 아니다.

최종 손실은 diagnose하면 component loss, abstain하면 λA다. 이 정의는 모든 답을 보류한 시스템이 selective error 0으로 우수해 보이는 문제를 막는다. 모든 episode를 보류하면 coverage는 0이고 terminal loss는 λA이며, diagnosed denominator가 없으므로 selective error는 NA다. NA를 0으로 대체하지 않는다.

이 연구의 "진단" 표현은 기록에서 annotation을 예측하는 선택적 출력이라는 뜻으로 제한한다. 부품을 분해해서 실제 원인을 확인한 것, 동시 고장의 인과 방향을 식별한 것, 수리를 선택하고 효과를 검증한 것이 아니다. 한 부품 상태 label이 있다고 해서 그 부품이 다른 신호 이상을 일으킨 물리적 원인이라는 주장까지 인증되지 않는다.

또한 이상 여부를 예측하는 과제와 이 네 head의 상태를 예측하는 과제는 다르다. SKAB의 anomaly/changepoint와 MetroPT의 제한된 정비 기록을 네 부품 정답으로 바꿀 수 없다. 별도 전이 평가를 설계할 수는 있지만 같은 성능표의 한 숫자로 합치지 않는다. 데이터가 바뀌면 정답의 의미·분모·관측 환경도 다시 선언해야 한다.

4. 변수와 상태: 확신도와 비용 상태를 분리한다

전체 상태를 s=(z,m,O,C,v,b,h_public)로 쓴다. z는 정책에 반환된 수치, m은 그 수치가 실제 관측되었는지 나타내는 mask다. O는 현재 episode에서 이미 확보한 원본 source identity의 집합이고, C는 이미 계산된 모듈 출력의 identity 집합이다. v는 공개된 availability이고, b는 남은 예산이다. h_public은 이후의 예측이나 비용 계산에 필요한 공개 실패·행동 이력이다.

z와 m을 나누는 것은 단순 구현 편의가 아니다. 압력 변화량 0은 실제로 변화가 없었다는 관측일 수 있다. 아직 압력 분석을 요청하지 않아 배열 자리만 0인 경우는 정보가 없다. 수치 0만 입력하면 두 경우가 같아지고, 어떤 값이 없다는 것 자체를 학습하는 과정도 불명확해진다. 관측 값과 mask를 같이 입력해 그 차이를 보존한다.

O와 C는 z보다 넓을 수 있다. 모듈 D가 내부에서 pressure_mean과 flow_mean을 실행한 뒤 조합 값 하나만 반환하면, 두 평균은 C에 존재하지만 정책 z에는 없다. 원본 압력·유량은 O에 있을 수 있다. 반대로 이미 원본이 O에 있어도 새 변환을 실행해야 하면 compute 비용은 발생한다. 캐시 재사용과 새 정보 획득을 같은 Boolean으로 표현하면 이 차이를 잃는다.

예측기는 qφ(y|z,m,status)로 표시한다. φ는 학습된 파라미터를 뜻하며 정책 비교 중 고정한다. 모듈이 새 값을 공개하면 qφ의 출력 확률은 달라질 수 있다. "예측기를 고정한다"는 것은 모든 시점에서 확신도가 같아야 한다는 뜻이 아니다. 같은 입력과 같은 mask라면 같은 확신도를 내는 같은 함수가 사용되어야 한다는 뜻이다.

한편 다음 분석의 가능한 결과를 예측하려면 별도의 pθ(o_a|s)가 필요하다. 여기서 o_a는 아직 요청하지 않은 action a의 반환값 또는 실패 결과다. qφ가 현재 y의 확률을 반환한다고 해서 아직 모르는 압력 slope의 분포까지 제공하지는 않는다. 이 두 분포를 혼동하면 VOI를 계산할 수 있는 것처럼 보이지만 실제로는 필요한 transition model이 비어 있다.

참조 fixture에서는 알려진 유한 joint world가 두 역할을 함께 제공한다. 각 세계에 모든 분석 값과 y가 들어 있으므로 관측과 일치하는 세계들에서 다음 값의 확률과 y의 확률을 계산한다. 산업 데이터에서는 그런 정확한 joint model을 갖고 있지 않다. 예측기 학습과 다음 관측 예측기의 학습은 다른 준비 항목이며 오류도 따로 평가해야 한다.

b는 모델의 확신도와 무관하게 action의 legality를 바꾼다. 같은 qφ라도 압력 창이 이미 읽혀 있으면 pressure_slope가 싸고, 처음 읽어야 한다면 비쌀 수 있다. 예산이 작은 시점에서는 이 차이로 feasible action set이 달라진다. 정책 state에 비용 정보가 필요한 이유는 확신도를 더 높게 만드는 것이 아니라 합법적인 continuation을 구별하기 위해서다.

그러나 상태에 변수를 추가했다고 자동으로 Markov성이 증명되는 것은 아니다. 미래 결과와 비용이 더 과거의 이력에 의존하면 그 이력을 충분히 표현해야 한다. 값에 의존하는 실패, 시간에 따른 센서 가용성 변화, shared-server contention이 있다면 단순 tuple 밖의 변수가 필요할 수 있다. 첫 모델에서는 그런 조건을 제외하거나 명시적인 상태·stratum으로 포함한다.

반대로 모든 요청이 성공하고, DAG와 source requirement가 결정론적이며, 같은 빈 캐시에서 시작하고, m이 요청 provenance를 완전히 담는다면 O와 C를 m에서 재구성할 수 있다. 이 경우 cache state는 정보상 중복이다. "모든 feature-only 정책은 불충분하다"는 주장 대신 재구성 가능한 경우와 실패 이력이 필요한 경우를 나눠 설명해야 한다.

5. Typed analysis DAG를 구체적으로 구성하기

analysis module은 정해진 입력에서 정해진 숫자를 반환하는 동결 함수다. 이 연구에서 skill이라는 표현을 사용할 때도 이런 분석 모듈의 계약을 뜻한다. 실행 도중 새로운 instruction을 생성하거나, 스킬을 병합하거나, 스킬 자체를 진화시키는 과정은 없다. 참가자가 실제 제출한 분석 skill은 현재 0개다.

모듈 계약에는 module ID와 version, raw source requirement, prerequisite module, 허용 창, 단위, 최소 또는 정확한 표본율, 출력 schema, 실패 schema, 구현 해시가 들어간다. pressure_mean은 압력 창 하나를 받아 bar 단위의 숫자를 반환할 수 있다. pressure_delta는 같은 창에서 마지막 값과 처음 값의 차이를 반환할 수 있다. 선형 trend는 시간축과 회귀 정의를 더 명시해야 하며, delta를 실제 시간당 slope라고 자동으로 부르면 안 된다.

v3의 pressure_slope라는 ID는 fixture 창의 끝점 차이를 계산한다. 그 단위도 bar_per_fixture_window로 제한되어 있다. 이를 Hydraulic의 60초 회귀 기울기와 같은 함수라고 해석하지 않는다. 산업 모듈을 만들 때는 평균 계산에 모든 유한 샘플을 쓰는지, std가 population인지 sample인지, slope의 시간 단위가 무엇인지 각각 고정한다. 작은 숫자 함수라도 정의가 바뀌면 predictor 입력의 의미가 바뀐다.

DAG는 모듈 간 선행 의존성을 나타낸다. 예를 들어 pressure_mean과 flow_mean이 먼저 계산되고, normalized_balance가 두 값을 사용한다고 하자. balance를 요청하면 서버는 필요한 source와 선행 모듈을 dependency closure로 찾는다. 이것이 transitive requirement다. 요청된 모듈의 직접 입력만 세면 선행 단계의 압력·유량 읽기와 계산이 빠질 수 있다.

노드	필요한 입력	출력	policy 공개 규칙
pressure_mean	동일 PS1 창	평균 압력	직접 요청했을 때 공개
pressure_delta	동일 PS1 창	끝점 차이	직접 요청했을 때 공개
flow_mean	동일 FS1 창	평균 유량	직접 요청했을 때 공개
normalized_balance	앞의 두 평균	fixture 조합 점수	balance만 요청했다면 조합 점수만 공개

v3 balance는 pressure_mean/10 − flow_mean/5라는 authored score다. 기준 숫자 10과 5는 fixture 정의다. 물리적 에너지 효율식이나 Hydraulic의 CE·CP 계산식이 아니다. 교육용 모듈을 실제 가상 센서로 재명명해서 사용하면 dependency 검증과 물리적 의미를 모두 잘못 전달하게 된다.

DAG 검사에는 cycle detection이 포함된다. A가 B를 필요로 하고 B가 A를 필요로 하면 실행 순서를 정할 수 없다. 없는 source를 요구하는 모듈, unit이 맞지 않는 연결, 다른 dataset version의 requirement도 거절한다. 단순히 Python 함수가 실행된다는 이유로 registry가 유효한 것은 아니다. 실행 전 계약 검사가 필요하다.

유용한 첫 모듈 은행은 mean, std, min, max, 사전 정의한 trend 정도로 작게 시작할 수 있다. 여러 feature를 추가하면 action space가 커지고 predictor와 value estimator를 학습할 데이터 요구도 커진다. 스펙트럼 band energy나 envelope를 넣으려면 원시 표본율·해상도와 수치 구현을 따로 검증한다. 저주파로 기록된 VS1을 고주파 bearing waveform처럼 처리하는 것은 같은 모듈 확장이 아니다.

모든 비교 정책에 같은 registry를 제공한다. 새 방법만 composite module을 받고 greedy는 atomic mean만 받으면 차이는 계획 능력과 도구 구성의 효과가 섞인다. 별도 tool-bank 실험을 할 수는 있지만 policy-only 주 실험과 구별한다. 도구 이름도 label을 직접 드러내는 이름을 쓰지 않으며, 전문가가 해석 가능한 물리량 이름의 공개 수준을 모든 정책에 맞춘다.

6. Source identity, 내부 캐시, 공개 증거의 보호 경계

source identity는 채널 이름 하나보다 자세해야 한다. 같은 PS1이라도 다른 episode의 창, 다른 시작·끝 위치, 다른 downsampling, 다른 파일 revision이면 같은 source가 아니다. 제안 identity는 archive hash, raw file hash, opaque episode ID, channel, [start,stop), sampling, unit, resolution, transform version을 포함한다. v3 identity보다 provenance 항목을 확장한 v5 설계이며 이미 모든 필드가 산업 실행에 구현되었다는 뜻은 아니다.

창은 반개구간 [start,stop)로 정의한다. start는 포함하고 stop은 제외한다. 예를 들어 100 Hz 주기의 [0,6000)은 6,000개 값이다. [0,3000)은 전반부이고 [3000,6000)은 후반부다. 값 개수와 decision time을 일관되게 확인할 수 있도록 정수 sample index와 원시 시간축의 대응을 보존한다.

캐시는 세 곳을 구분한다. raw cache는 확보한 source buffer를 보관한다. module cache는 이미 계산한 수치 출력을 보관한다. disclosed evidence는 정책이 실제 전달받은 수치와 ID를 보관한다. raw cache와 module cache의 존재를 알고 있다는 것은 그 값 자체를 알고 있다는 뜻이 아니다. 정책에 필요한 것은 다음 조회의 정확한 quote와 합법적인 공개 요청이다.

예를 들어 balance 요청으로 pressure_mean이 내부 계산된 뒤 pressure_mean을 따로 요청하면 source와 계산은 재사용할 수 있다. 추가 evidence 비용이 0일 수 있지만 그 요청 이전의 predictor 입력에 pressure_mean을 넣으면 안 된다. 추가 정보가 공개되는 시점이 명확해야 한다. 반환 오버헤드는 실제 request_ns 등에 기록하고 0이라는 normalized 비용과 별도로 본다.

부분 겹침도 주의한다. [0,3000)과 [0,6000)은 일부 원시 값이 겹치지만 정확히 같은 key가 아니다. 현재 제안의 기본 규칙은 exact-key 재사용이다. interval union 캐시를 따로 구현·검증하지 않았다면 두 번째 창 전체를 새 key 비용으로 청구한다. 반대로 겹치는 값마다 자동 할인했다고 주장하려면 읽은 구간 합집합과 변환 비용을 재현할 수 있어야 한다. 두 규칙을 실행 중에 임의로 섞지 않는다.

정답은 evaluator 전용 label 저장소에 둔다. 정책이 profile.txt, 안정성 flag, 조건을 암시하는 파일명, 미래 샘플을 요구하면 값 없이 거절한다. module의 오류 문자열에도 수치를 그대로 넣지 않는다. exception에 숨겨진 raw 값이나 원본 condition code가 포함되면 숫자 output schema를 지켜도 정보 경계가 깨진다.

파일 hash 검증은 바이트가 예상 원본과 같은지 확인한다. semantic verification은 그 바이트에서 다시 class·row key·요약을 계산해 derived output과 맞는지 확인한다. profile target을 999로 바꾸고 output hash와 seal을 다시 써도 허용 class 검사로 거절해야 한다. 안정성 count나 transition summary도 재계산해야 한다.

하지만 모든 원본과 예상 hash를 함께 다른 합법 자료로 바꾸는 공격은 로컬 self-seal만으로 인증할 수 없다. 외부에서 고정한 expected hash나 신뢰할 원본 연결이 필요하다. 이 연구의 unsigned manifest는 무결성·재현성 기록이지 publisher signature가 아니다. 코드의 밑줄 private 속성도 적대적인 in-process Python을 막는 보안 경계가 아니다.

현재 evidence service와 evaluator는 신뢰하는 같은 process 안의 역할 분리다. 미래 외부 LLM이나 임의 코드를 받아 실행하려면 별도 서비스의 접근 권한·실행 격리·resource cap이 필요할 수 있다. 현재 문서가 그 배포를 수행하거나 완료를 인증하지 않는다. 첫 비교의 정책은 승인된 named action만 호출하는 trusted client로 한정한다.

7. 비분리가능 비용과 실제 자원 ledger

전체 비용은 action 이름별 상수만으로 설명되지 않을 수 있다. 분석 a의 transitive source 집합을 R(a), 필요한 모듈의 dependency closure를 D(a)라고 하자. 새 action의 charge는 R(a) 중 O에 없는 source 읽기와 D(a) 중 C에 없는 계산을 합한다. 이미 확보된 exact identity의 읽기를 또 청구하지 않는다는 의미에서 state-dependent marginal cost다.

두 분석을 합한 비용이 각각의 standalone 비용 합과 같지 않으므로 비분리가능이라고 부른다. 이것은 새 수학적 비용 개념을 발명했다는 뜻이 아니다. 비교 실험에서 "한 feature는 한 번에 한 비용"이라는 단순 입력 모델과 실제 dependency executor를 같은 것으로 취급하지 말자는 뜻이다.

v3의 직접 계산은 pressure read 6, mean compute 1, delta compute 2를 사용한다. pressure_mean을 먼저 실행하면 7을 쓰고, 같은 source의 delta는 새 계산 2만 사용한다. 합계는 9다. raw 읽기를 다시 붙이면 (6+1)+(6+2)=15가 된다. 9와 15의 차이는 cache 회계 검사다. 정책이 어떤 episode에서 더 좋은 정답을 냈다는 결과도, 실제 sensor power가 줄었다는 결과도 아니다.

실제 기록 형식에 가까운 다른 교육용 예를 보자. 분석 A는 PS1 6,000개와 FS1 600개를 요구하고, 분석 B는 동일 PS1 6,000개와 TS1 60개를 요구한다. standalone 합은 6,600+6,060=12,660개 값이다.

exact PS1 창을 공유하면 source count는 6,000+600+60=6,660이다. 동일한 raw input을 두 번 세는지 확인하는 계산이며 실제 action bank 또는 모델 성능을 실행한 것이 아니다.

이때 6,660은 논리적으로 승인한 센서 값 개수다. ZIP의 압축 byte 수도, CSV 원문 읽기 byte 수도, 네트워크 전송량도 아니다. 숫자를 float64로 보내면 payload 단위는 또 달라지고, 텍스트로 보내면 자릿수·구분자에 따라 크기가 달라진다. 결과가 숫자 summary만 반환되는 분석에서는 서버 내부 raw read와 정책에게 전달된 payload도 구별해야 한다.

따라서 resource ledger는 승인한 source key와 값 개수, 실제 file I/O를 계측했다면 그 byte 수, parsing 시간, module 계산 시간, cache retrieval 시간, predictor 시간, policy planning 시간을 따로 저장한다. 계측하지 않은 값은 null 또는 unmeasured로 남긴다. v3의 encoded source byte는 in-memory slice의 encoding 크기이며 OS나 network I/O를 측정한 값이 아니다.

cold/warm을 말할 때도 어떤 캐시인지 지정한다. 논리적 episode cache가 비어 있어도 OS page cache는 warm일 수 있다. 프로그램을 새로 띄운 것이 디스크가 cold라는 보장은 없다. 모듈 output을 사전 계산한 warm run에는 prewarm 비용과 공개 상태를 기록해야 한다. 비교 정책 하나만 warm buffer를 사용하는 결과를 total-system 효율 비교에 넣으면 안 된다.

실패 비용은 성공 비용과 같은 ledger에서 유지한다. 압력 읽기에 4를 썼고 분석을 시작해 compute 1을 쓴 뒤 실패했다면 5는 사라지지 않는다. 다음 pressure 분석이 raw cache를 재사용할 수도 있다. 선행 단계에서 얻은 수치가 전혀 공개되지 않았더라도 자원 사용은 발생했다. retry를 자동으로 무료 처리하면 실패가 많은 정책의 비용을 낮게 만들게 된다.

사전 거절은 별개다. 없는 prerequisite나 이미 알려진 예산 초과를 실행 전에 발견하면 acquisition charge는 0일 수 있다. 그래도 요청을 해석하고 검사한 시간이 들었으므로 request overhead는 보존한다. "거절 0 비용"을 전체 계산 비용 0으로 읽으면 안 된다. 로그의 charge와 elapsed field가 서로 다른 단위를 가진 이유다.

첫 산업 strict budget의 후보는 새 exact source key가 요청한 값 개수다. 실제 compute와 planning 시간은 따로 보고한다. 시간을 hard budget에 넣고 싶다면 사전 upper bound나 reservation, 초과 시 중단과 비용 처리 규칙을 추가해야 한다. action의 realized wall time은 요청하기 전에 정확히 알 수 없으므로 측정한 시간만 보고 strict cap이 보장되었다고 말할 수 없다.

그 가중치·상한·초기 evidence 포함 여부는 현재 미정이다. 정규화된 비용 하나를 만들더라도 원시 요소를 함께 보고하고 가중치 민감도를 확인한다. 저장된 기록의 조회 비용에서 실제 설비 정지 시간, 유지보수 가격, 센서 에너지 비용으로 넘어가는 추론은 별도 측정이 필요한 연구다.

8. Masked predictor를 단계별로 만드는 방법

첫 산업 predictor 후보는 네 head별 multinomial logistic regression처럼 단순한 수치 모델이다. 새로운 architecture의 기여를 주장하려는 선택이 아니다. 작고 재현 가능한 baseline을 먼저 두어 정책이 예측기 용량을 바꿔 이기는 상황을 줄이려는 설계다. 현재 이 모델을 구현하거나 학습하지 않았다.

먼저 train group만 사용해 각 수치 feature의 scaling 통계를 만든다. mean과 scale을 train에서 고정한 뒤 dev/calibration/final에 그대로 적용한다. final 전체를 보고 표준화하면 예측값을 쓰지 않았더라도 분포 정보를 학습에 가져온다. unit 변환도 모듈 계약에서 고정하며 서로 다른 물리량을 의미 없이 같은 숫자 범위로 혼합하지 않는다.

입력 vector에는 scaled disclosed value, 관측 mask, 사전에 허용한 status indicator를 이어 붙인다. 아직 보지 않은 값 자리에 0을 넣을 수 있지만 그 0을 실제 관측으로 읽지 않도록 mask를 반드시 붙인다. 수치 실패와 미요청을 같은 상태로 만들지도 않는다. 실패가 값과 관련될 수 있으면 그 공개 status를 어떻게 학습할지 선언한다.

학습 mask는 registry에서 가능한 행동 이력으로 만든다. raw input을 임의로 숨긴 arbitrary mask는 DAG·공개 계약과 맞지 않을 수 있다. composite를 요청했지만 선행 평균은 공개하지 않은 mask, 선행 raw는 캐시되었지만 값은 실패한 상태를 표현해야 한다. random prefix와 정책 trajectory를 섞는다면 비율과 sampling seed를 dev 전에 선언한다.

네 head는 서로 다른 class map을 가진다. 출력으로 cooler 3개, valve 4개, leakage 3개, accumulator 4개 확률을 내고 각 head argmax를 point prediction으로 삼는다. 학습 목적함수는 softmax cross-entropy 후보지만 첫 정책 채점은 네 0/1 error 평균이다. 학습 loss와 최종 평가 loss가 같은 이름으로 섞이지 않게 기록한다.

현재 수치 score의 후보는 네 head에서 1−max probability를 평균한 값이다. 올바른 확률이면 mean component error의 기대값과 연결되지만 실제 학습 확률은 틀릴 수 있다. 이 score를 calibration 없이 진짜 error probability라고 단정하지 않는다. confidence mapping과 terminal gate도 정책 비교 전에 고정해야 한다.

네 head 최대 확률을 곱해서 vector correctness 확률로 부르는 것은 추가 가정이 필요하다. head들이 같은 pressure trace에 의존하므로 독립이라고 볼 근거가 없다. 평균 component risk와 any-head vector risk의 차이를 유지하고, vector confidence를 쓰려면 joint 모델이나 별도 검증 절차를 명시한다. calibrated marginal이 있다는 주장만으로 joint correctness가 인증되지 않는다.

정책을 학습하기 전 predictor φ를 동결한다. policy training에서 predictor weights를 같이 바꾸면 policy-only 실험이 아니다. 그런 end-to-end system을 별도로 평가할 수는 있지만 네트워크·학습 비용·confidence calibration이 다른 전체 시스템 비교라고 표시한다. 기존 방법의 native predictor를 그대로 두는 대조 역시 별도 표다.

complete-bank teacher는 모든 허용 feature를 본 참고 예측기다. 제한 bank와 partial-input predictor의 표현 차이를 이해하는 데 도움이 될 수 있지만 항상 더 정확하다는 보장은 없다. 더 많은 입력이 과적합을 늘릴 수도 있고 train/test shift가 달라질 수도 있다. 따라서 teacher line을 mathematical upper bound라고 쓰지 않는다.

마지막으로 failure/status, 초기 관측 packet, availability mask를 바꿔도 같은 함수가 쓰이는지 확인한다. 실제 hidden label만 바꿨는데 policy 선택이나 confidence가 달라진다면 누출이다. 고정해야 할 것은 함수와 입력 권한이며, 새로운 관측 뒤에 값이 달라지는 정상적인 확률 업데이트를 금지하는 것이 아니다.

9. 같은 관측값인데 다음 행동이 달라지는 조건부 반례

이 절의 반례는 학습 결과가 아니라 state representation의 논리 검사다. 목표는 두 history가 같은 z·m·b를 갖는데 다음 action의 비용과 legality가 다른 경우를 만드는 것이다. 그러면 z·m·b만 입력하는 memoryless 정책은 두 상태를 구별하지 못한다. 전체 과거 이력을 기억하는 정책이나 정확한 quote 정보를 받은 정책까지 모두 불가능하다는 뜻은 아니다.

가상 환경에서 압력 source P와 유량 source F의 read 비용을 각각 4로 둔다. 실패 probe의 compute 비용은 1이다. 초기 예산은 8이다. pressure_probe와 flow_probe는 source를 읽고 계산을 시작한 뒤, 값과 무관한 이유로 실패한다. 실패 여부가 hidden label이나 raw 값에 대한 추가 정보를 주지 않는다고 가정한다.

history H_P에서는 pressure_probe를 실행했다. P를 읽어 캐시했으므로 O={P}이고 4+1=5를 썼다. 하지만 수치 출력은 반환되지 않아 z는 비어 있고 m은 모두 0이다. 남은 b는 8−5=3이다. history H_F에서는 flow_probe를 실행해 O={F}, 사용 비용 5, 빈 z와 같은 m, b=3이 된다. public availability도 같게 둔다.

상태	공개 수치·mask	source cache	남은 예산	다음 pressure 분석 비용
H_P	empty, all-zero	P	3	compute 1
H_F	empty, all-zero	F	3	read 4 + compute 1 = 5

다음 pressure_mean은 P를 읽어 한 숫자를 반환하는 action이고 compute가 1이다. H_P에서는 P가 이미 있으므로 비용 1이며 실행할 수 있다. H_F에서는 5가 필요해 남은 3으로 실행할 수 없다. 동일한 z·m·b, 동일한 frozen predictor confidence인데 feasible set이 달라진다.

더 명시적으로 hidden target이 압력 값에서 완전히 결정되고 prior에서는 두 target이 반반이라고 두자. 실패는 정보가 없으므로 두 상태의 model risk는 같다. λA=0.3이면 둘 다 아무 값 없이 종료할 때 abstain한다. H_P는 pressure_mean을 요청해 risk를 0으로 만들 수 있지만 H_F는 요청이 불가능하다. 이 가정 아래 최적 행동은 두 상태에서 다르다.

이 반례의 read 4·4는 equal-spend를 맞춘 교육용 설정이다. v3 default pressure 6·flow 5를 그대로 실행한 실제 결과라고 표시하지 않는다. v3에서 실패 뒤 raw cache와 비용을 유지하는 mechanics는 구현되어 있지만, 이 두 history의 숫자 대비는 별도의 formal construction이다. source와 predictor를 직접 구현해 새 성능표를 만든 것이 아니다.

복구 방법은 여러 가지다. 정책에 O와 C를 제공할 수 있다. 모든 action의 정확한 marginal quote를 제공하고 history에서 cache 상태를 재구성할 수도 있다. 마지막 실패 action과 필요한 provenance를 기억하는 recurrent representation도 후보가 된다. 선택한 representation이 미래 비용·관측 분포를

결정하는지 확인해야 하며 "항상 명시적인 cache vector를 추가해야만 한다"는 주장을 하지는 않는다.

성공만 있는 고정 DAG에서는 왜 반례가 사라질 수 있을까. 같은 빈 cache에서 시작하고 요청한 module ID와 창이 m에 모두 기록된다면 O는 요청한 action들의 raw dependency 합집합이다. C도 그 dependency closure의 합집합이다. 실행 순서가 같지 않아도 같은 요청 집합이면 최종 cache를 재구성할 수 있다. 이 제한 조건에서 cache representation은 계산 편의를 줄 수 있지만 추가 정보는 아니다.

failure가 값에 의존하면 이 절의 predictor 동일성 가정도 바뀐다. 압력이 너무 높아서 계산이 실패했다는 status 자체가 정보가 될 수 있다. 그때는 status를 숨겨서 같은 prior라고 말할 수 없다. 공개 실패 결과를 qφ와 outcome model에 포함하거나 해당 stratum을 따로 다룬다. 정보와 비용을 모두 설명해야 올바른 Markov claim이 된다.

10. XOR가 보여주는 보완 정보와 greedy의 정확한 한계

X1과 X2가 서로 독립이고 각각 0 또는 1을 같은 확률로 갖는 가상 세계를 만든다. 정답 Y는 두 값이 다르면 1, 같으면 0인 XOR다. 네 world의 prior는 각각 1/4다. 이는 Hydraulic label을 모델링한 것이 아니라 정보 보완성의 계산 가능한 최소 예시다.

X1	X2	Y	prior
0	0	0	1/4
0	1	1	1/4
1	0	1	1/4
1	1	0	1/4

아무것도 보지 않으면 Y는 반반이다. X1=0을 보아도 가능한 world는 처음 두 행이므로 Y는 여전히 반반이다. X1=1에서도 마찬가지다. X2 하나만 보아도 같다. 둘을 함께 보면 Y가 완전히 정해진다. entropy 단위 bit에서 H(Y)=1, I(Y;X1)=I(Y;X2)=0, I(Y;X2|X1)=1이다.

무조건부 marginal information이 0인 측정이 이미 다른 측정을 한 뒤에는 1 bit의 가치가 생긴다. adaptive diminishing returns를 이 정보 목적에 당연히 적용할 수 없는 이유다. adaptive submodularity 원문 (https://jair.org/index.php/jair/article/view/10731)의 greedy 보장은 해당 property를 만족하는 목적·관측 모델을 전제로 한다. XOR 계산은 그 가정을 검증하지 않고 보장 문장을 가져오면 안 된다는 반례다.

terminal loss로도 같은 현상을 볼 수 있다. binary 진단에서 Y를 찍으면 model expected error는 0.5다. λA=0.3으로 보류할 수 있으면 최선의 현재 terminal risk ρ는 min(0.5,0.3)=0.3이다. X1 또는 X2 하나를 얻어도 여전히 0.3이다. 둘을 얻으면 risk가 0이다.

각 query 비용이 1, 예산이 2이면 pair acquisition은 feasible하다. 한 단계 expected terminal gain이 양수인 경우만 선택하는 greedy는 두 candidate의 gain이 0이므로 처음에 멈춘다. 그러나 두 행동을 고려하는 정책은 loss를 0.3에서 0으로 낮출 수 있다. 이 계산은 "나중에 정보가 생긴다"는 말보다 정확하다. cost·prior·terminal gate·budget을 명시했기 때문이다.

이 예시에는 알고리즘 규칙이 포함되어 있다. zero-gain action을 강제로 탐색하는 greedy, 미리 조합 action을 등록한 정책, 두 단계 lookahead는 해결할 수 있다. 따라서 모든 myopic method가 같은 결과를 내는 것이 아니며 구현의 tie와 stopping rule을 기록해야 한다. baseline을 약하게 만들어 놓고 반례가 있다고 우월성을 주장하지 않는다.

zero-gain stop만의 문제인지 확인하려면 Z라는 proxy를 추가할 수 있다. E가 독립 Bernoulli(0.2)이고 Z=Y XOR E라면 Z는 정답을 80% 확률로 맞추는 noisy label proxy다. query 비용은 동일하게 1이고 예산은 여전히 2다. Z를 관측하면 binary Bayes error는 0.2이므로 terminal gain은 0.3−0.2=0.1이다. X1과 X2의 첫 gain은 0이다.

즉시 gain greedy는 Z를 먼저 선택한다. 예산은 1만 남아 X1과 X2 둘을 모두 얻을 수 없다. Z와 X1을 같이 봐도 Y에 대한 posterior error는 0.2다. X1만으로 Y 정보가 없고, 그 조건에서도 noisy proxy가 주는 확률은 그대로이기 때문이다. 따라서 최선 종료 risk는 0.2다. 처음부터 X1→X2를 선택한 pair 정책은 0이다.

선택 비용을 목적함수에도 더하는 예를 보자. α=0.05로 두면 Z 후 종료의 objective는 0.2+0.05×1=0.25, XOR pair는 0+0.05×2=0.10, 처음 보류는 0.3이다. 단순 stop tie만 바꿔서 해결할 수 없는 유한 예시다. 여기의 α와 λA는 가상 정의이며 산업의 useful effect size가 아니다.

네 head 과제에 이 아이디어를 넣을 수도 있다. 다른 세 head가 이미 정확히 알려져 있고 하나만 XOR라고 두면 unresolved component error는 0.5/4=0.125다. λA를 0.1로 두면 현재 terminal risk는 0.1이고 한 bit 뒤에도 같다. pair 뒤에는 0이다. 이 embedding은 loss scale을 확인하는 계산이지 UCI의 네 class 구조가 실제 XOR라는 증거가 아니다.

또한 information gain과 decision gain을 같은 것으로 보면 안 된다. entropy는 전체 posterior 불확실성을 재고, terminal risk는 선택한 loss와 보류 비용에 따라 달라진다. 정보가 늘어도 argmax가 바뀌지 않거나 여전히 보류하는 구간에서는 0/1 terminal gain이 0일 수 있다. MI-greedy와 loss-greedy는 서로 다른 baseline이며 같은 이름으로 묶지 않는다.

선행연구의 non-myopic AFA와 BRiG-AFA (https://arxiv.org/html/2608.02305v1)는 이미 context와 후속 측정의 가치를 다룬다. ProbeFlow의 XOR는 novelty 증거가 아니다. 공유 cost와 공개 상태를 정확히 넣은 인터페이스에서도 짧은 continuation이 필요한 조건을 명료하게 설명하고, 산업 데이터에서 그런 조건이 존재하는지 반증 가능한 평가를 설계하는 도구다.

11. 제한된 Bellman 해와 stopping certificate

finite reference에서는 latent world의 확률과 모든 action outcome이 알려져 있다고 가정한다. 가능한 반환값이 유한하고, state가 이후 transition과 비용을 결정하며, action별 실제 charge가 결정론적 비음수여야 한다. 아직 공개하지 않은 새 action의 수를 제한해 recursion이 종료되도록 한다. cached output의 charge가 0이라도 같은 값을 무한히 다시 요청하는 branch는 허용하지 않는다.

현재 terminal 선택의 model risk를 ρ(s)라 하고 남은 예산을 b라 하자. action a를 선택하면 outcome o가 나와 successor s′가 되고 b′=b−c(a|s)가 된다. α는 evidence cost를 objective에도 넣는 비음수 가중치다.

V(s,b)=min{ρ(s), min_a [α·c(a|s)+E_o V(s′,b′)]}

밖의 min은 지금 종료하는 것과 더 정보를 얻는 것의 비교다. 안의 min은 현재 feasible action 중 최선의 continuation을 고른다. expectation은 숨겨진 실제 world를 알고 선택하는 것이 아니라 현재 공개 정보에 조건부로 가능한 outcome들의 확률을 사용한다.

finite horizon에서 이 식이 맞는 이유는 뒤에서 앞으로 생각하면 간단하다. 더 이상 행동할 수 없으면 V는 ρ다. 한 행동이 남으면 각 outcome에서 종료 risk를 평균하고 action 비용을 더해 현재 종료와 비교한다. 두 행동이 남으면 이미 구한 한 행동 값으로 같은 계산을 한다. state와 확률·비용 정의가 올바르면 이 induction이 모든 남은 horizon에 적용된다.

v3 exact reference는 α=0의 budgeted expected terminal loss를 최우선으로 낮춘다. loss가 같을 때만 expected evidence cost를 tie-break로 쓴다. 이는 arbitrary α>0를 더한 목적과 다르다. kernel의 정책 설명에 실제 구현된 목적함수를 적어야 하며 cost-sensitive optimality를 더 강하게 표현하면 안 된다.

v3는 최대 8개 action과 32개 world를 허용하며 기본 fixture는 16개 world다. 전부 분기하는 exact planner는 action과 outcome이 늘면 급격히 커진다. 실제 연속 센서 요약, 여러 window, 실패 확률을 가진 산업 상황에 같은 runtime 보장을 이식할 수 없다. 작은 표준 모델에서 reference optimum을 계산하는 역할로 제한한다.

정확한 transition distribution도 별도 요구다. classifier가 현재 class probability를 내도 미관측 action의 결과 분포는 알 수 없다. 잘못된 pθ로 계산한 V는 그 추정 모델에 대한 plan이지 실제 산업 최적값이 아니다. 직접 risk-to-go regression을 쓰는 방법도 학습 target과 approximation error가 있으며 Bellman 형태를 썼다는 이유로 exact certificate가 생기지 않는다.

제한된 lower bound는 free information relaxation으로 만들 수 있다. 현재 이용 가능한 모든 evidence를 비용 없이 공개하고, 그 정보를 사용하거나 무시할 수 있는 Bayes decision rule을 허용한다. 그 뒤의 최선 terminal loss 기대값을 L_free(s)라 하자. 이 relaxation은 원래 문제보다 행동·비용 제약이 약하므로 L_free≤V≤ρ다. 오른쪽 부등식은 원래 정책이 언제든 현재 종료할 수 있기 때문이다.

따라서 ρ−L_free≤ε이면 지금 종료하는 decision은 이 모델의 최적 continuation보다 expected objective가 ε 이상 나쁠 수 없다. 이것이 stopping certificate의 정확한 의미다. 실제 episode의 답이 틀리지 않는다는 보장, calibration coverage 보장, 새로운 장치에서의 안전 보장이 아니다.

XOR에서 두 bit가 모두 available이면 L_free=0이고 초기 ρ=0.3이다. gap이 커서 작은 ε certificate로 종료를 인증할 수 없다. 한 bit가 unavailable이고 남은 모든 evidence가 Y를 구별하지 못하면

L_free=0.3이다. 현재 ρ와 같아 종료를 인증할 수 있다. 이 예시는 availability와 정보 제약이 certificate를 바꾼다는 것을 보여준다.

free-information Bayes rule과 all-input teacher는 다르다. teacher는 하나의 고정 learned model이며 더 많은 입력에서 실제 오차가 올라갈 수 있다. Bayes relaxation은 additional information을 무시하는 결정도 허용하고 정확한 joint prior를 전제로 한다. teacher score를 L_free 자리에 넣으면 부등식이 보장되지 않는다.

실패로 실제 charge가 outcome마다 달라지는 경우도 현재 단순식의 가정 밖이다. finite failure model을 정확히 안다면 outcome별 charge와 reservation 상한을 가진 더 일반식으로 확장할 수 있다. 산업 wall time은 그런 확정 상한을 갖고 있지 않으므로 같은 hard-budget proof를 주장하지 않는다. 알려지지 않은 가정은 contract에서 unknown으로 남긴다.

12. 실제로 만들 파이프라인과 단계별 도구·입출력

파이프라인은 원시 파일 준비와 학습·평가를 구분해 구성한다. 다음 순서는 이미 전체 구현이 완료되었다는 목록이 아니라, 현재 준비된 원본에서 실험으로 넘어가기 위한 입력·산출물 계약이다. 기존 v3/v4를 덮어쓰지 않고 매 실행에 새 output directory를 사용한다.

첫 단계는 local intake다. 도구는 기존 표준 라이브러리 파일 reader와 hash verifier다. 입력은 보존된 TSV/profile, 확보 영수증, 파일 manifest다. 출력은 raw_inventory와 parsing audit다. 원본 URL·시점·byte 수·SHA-256을 유지하고 파일 path가 지정 root 안에 있는지 확인한다. 문서에 있는 설치 명령이나 archive 안의 코드는 실행하지 않는다.

두 번째는 schema와 label 분리다. 센서 파일은 행/열·finite value·시간축·단위를 검사한다. profile은 허용 class·다섯 번째 flag·행 번호를 검사한다. 출력은 private labels와 opaque episode inventory다. policy view에는 label-bearing original row name을 넣지 않는다. label store의 권한과 센서 source store의 권한을 분리한다.

세 번째는 group audit와 split이다. 입력은 실제 experiment block 또는 운전·시험 이력 sidecar다. 출력은 train/dev/calibration/final의 group list와 hash다. 독립 group 출처가 없으면 independent evaluation branch는 blocked다. 파일을 읽을 수 있다는 이유로 임의 random-cycle split을 자동 생성하지 않는다.

네 번째는 module freeze다. mean/std/trend 등 수치 함수와 DAG를 동결하고 unit/window/sampling을 검증한다. 출력은 registry.json과 구현 hash, 허용 action 목록이다. train 자료에서 모든 허용 분석 결과를 미리 계산할 수 있지만 그 offline training cost는 기록한다. final policy에는 미리 계산한 전체 feature를 공개하지 않는다.

다섯 번째는 predictor fitting이다. 출력은 train-only scaling, class map, feature order, φ weights와 training-mask definition이다. 외부 LLM이 아닌 수치 predictor가 네 categorical probability를 반환한다. 모델 선택은 dev에서 하고 final에는 접근하지 않는다. 새 라이브러리가 필요하다면 그 도입을 별도로 결정해야 하며 현재 task에서 설치하거나 학습하지 않는다.

여섯 번째는 acquisition policy의 dev 선택이다. 입력은 frozen predictor, typed registry, train-only history와 후보 policy다. 출력은 실제 quote를 사용하는 행동 선택 규칙과 stop/gate configuration이다. next-outcome model을 쓰는 경우 pθ artifact도 별도 저장한다. 단순 greedy, pair lookahead, strong AFA adapter를 같은 권한에서 비교한다.

일곱 번째는 calibration이다. dev 선택을 끝낸 정책과 predictor를 별도 calibration group에 실행해 score·mask·terminal outcome을 얻는다. threshold나 score mapping을 fitting하는 경우 범위와 population 가정을 기록한다. 이를 final prediction accuracy 실험으로 보고하지 않는다. 어떤 정책의 mask 분포에 대해 calibration했는지도 기록한다.

여덟 번째는 final runner다. executor가 매 action을 검증하고 실제 ledger를 갱신한다. predictor는 공개값만 읽으며 evaluator는 terminal artifact가 확정된 뒤 hidden label과 비교한다. 출력은 action record, terminal record, resource record, summary와 manifest다. hidden labels를 보고 실패 action을 재선택하는 retry는 허용되지 않는다.

마지막으로 verifier는 hash뿐 아니라 metric denominator, state transition, source reuse와 공개 mask를 재계산한다. nondeterministic timing은 기록을 보존하고 동일 숫자로 replay되었다고 주장하지 않는다. software verification과 과학적 타당성은 별개이므로 verification PASS를 진단 성공으로 표시하지 않는다.

이 구조의 optional LLM은 여섯째와 여덟째 단계에서 action proposal을 낼 수 있다. calibration threshold를 임의로 승인하거나 숫자 predictor를 대신하는 역할이 아니다. LLM의 호출 ID·전송 입력·token/cost/time 한도가 없는 현재 상태에서는 실제 invocation을 실행하지 않는다.

13. 정책·예상 관측값·LLM의 역할을 기술적으로 나눈다

greedy_terminal_gain의 후보 정의는 현재 model terminal risk ρ(s)에서 action a를 실행한 뒤의 기대 terminal risk를 뺀 값이다. Δ(a|s)=ρ(s)−E_o ρ(T(s,a,o))로 쓰고 이를 실제 marginal cost로 나눠 candidate를 고를 수 있다. 현재 quote가 0이면 양의 gain에 대한 순위 규칙을 별도로 정한다. 어떤 tie와 tolerance, zero-gain stop 규칙을 쓰는지 기록해야 한다.

이 기대값을 얻는 방법은 중요한 별도 도구다. authored finite model에서는 관측과 맞는 world들의 candidate output을 세면 된다. 산업 모델에서는 hidden outcome을 알고 선택하면 안 된다. train-only episode들의 outcome을 사용한 empirical conditional model이나 direct risk regression을 만들어야 한다. qφ의 현재 confidence만으로 기대 improvement를 정확히 계산한 것으로 표시하지 않는다.

한 가지 단순 후보는 training feature bank의 비슷한 observed episode들을 찾아 candidate output의 분포를 근사하는 것이다. distance는 현재 실제 공개된 feature와 mask에서만 계산하고 class label로 이웃을 고르지 않는다. 이웃들의 아직 미관측 candidate output은 학습 자료의 counterfactual target이므로 offline estimator fitting에 사용할 수 있지만, 현재 final episode의 값을 조회해서는 안 된다.

연속값에서 관측값의 완전 일치를 요구하면 이웃이 하나도 없을 수 있다. distance scaling, neighborhood 크기, smoothing을 dev에서 선택하고 forecast 검증을 해야 한다. misspecified predictor나 outcome estimator는 잘못된 VOI를 만들 수 있다. expected gain이 높았다는 policy 로그는 실제 gain의 증거가 아니므로 realized loss·cost와 따로 저장한다.

shared source도 forecast에 일관성을 요구한다. 같은 raw pressure 창에서 나온 mean과 delta를 서로 독립적으로 sampling하면 실제로 존재할 수 없는 조합을 만들 수 있다. 공동 raw-derived outcome이나 coherent training record를 함께 사용해야 한다. 공개되지 않은 prerequisite를 estimator의 현재 입력으로 넣는 것은 forecast를 잘하는 문제가 아니라 누출이다.

pair lookahead는 첫 action의 즉시 risk 대신 두 번째 feasible continuation을 포함한다. 각 first outcome에서 다음 action이 달라질 수 있으므로 고정 pair 점수와 adaptive two-step plan을 구별한다. 단순 pair enumeration은 action 수가 M이면 대략 M^2의 candidate 조합을 고려할 수 있고 outcome branch 수에 따라 더 비싸진다. 그 계산은 planning ledger에 포함한다.

risk-to-go를 직접 회귀하는 후보도 있다. frozen predictor를 train episode의 partial history에 적용하고, terminal label loss 또는 짧은 continuation target을 regression한다. BRiG-AFA의 공식 formulation (https://arxiv.org/html/2608.02305v1)은 이미 이런 방향을 다룬다. ProbeFlow adapter는 cache/history·nonuniform actual charges·abstain을 추가할 수 있지만 그런 변경은 native reproduction이 아닌 adaptation이다.

BRiG의 unit-cost exact-count setting을 그대로 가져와서 6,000-value 압력과 60-value 온도를 같은 query 1로 청구하면 primary budget이 바뀐다. NM-PPG의 additive feature objective를 DAG executor에 연결하는 경우에도 state와 gradient/rollout의 정의가 달라진다. 대조군의 실패를 구현 차이로 만들지 않도록 adapter의 action grouping·stop·cost·training target을 기록한다.

optional LLM은 이런 수치 estimator와 별개의 action proposal client다. 입력은 허용 registry, DAG 설명, 이미 반환된 evidence와 ID, 현재 fixed-predictor 출력, source/cache identity와 quote, availability, remaining budget이다. raw array, label, profile, 아직 공개되지 않은 derived output과 final test feedback은 받지 않는다.

출력은 allowlisted module ID 하나 또는 stop/abstain 제안처럼 작게 고정할 수 있다. 예를 들어 {"action":"pressure_delta"}라는 구조를 server validator가 확인한다. LLM이 설명문에 다른 작업을 섞거나 존재하지 않는 module ID를 생성하면 실행하지 않는다. rationale을 기록할 수는 있지만 정답·근거 인증·calibration 결과로 사용하지 않는다.

LLM이 diagnose를 제안해도 실제 네 class prediction은 동일 frozen predictor가 만든다. 그래야 language model의 지식이나 수치 추론 성능이 policy-only 비교를 바꾸지 않는다. LLM 자체로 조건을 예측하는 전체 시스템은 별도 실험이며 모델의 입력 정보·토큰·비용·지연을 다시 맞춰야 한다.

실제 invocation 전에 model ID와 revision, endpoint, 전송 데이터, prompt hash, 호출·token·cost·time 상한과 retry 규칙을 고정해야 한다. 현재 그런 live 실행은 없다. tool call이 성공했다고 LLM 정책이 유효한 진단을 완료한 것도 아니다. legality는 executor, 숫자 예측은 predictor, 보정은 calibration 도구, 최종 채점은 evaluator가 담당한다.

14. 보류·calibration·adaptive mask의 선택 편향

보류는 정보가 부족할 때 답을 내지 않는 terminal action이다. 이것은 실패를 숨길 방법이 아니라 loss와 coverage를 함께 계산하는 선택이다. 현재 mask에서 model mean component risk가 λA보다 낮으면 diagnose를 허용하는 단순 rule을 쓸 수 있다. 하지만 model risk라는 숫자는 실제 error probability의 인증이 아니다.

Selective classification 문헌 (https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html)은 어떤 입력을 답하고 어떤 입력을 거절할지 선택하며 risk와 coverage를 같이 다룬다. 이 연구도 그 관점을 따른다. 논문에서 risk를 낮췄다고 말할 때 coverage가 얼마나 줄었는지 숨기면 안 된다. 쉬운 episode 몇 개만 답하는 시스템은 낮은 selective error와 매우 낮은 coverage를 가질 수 있다.

calibration은 score와 실제 error의 관계를 별도 자료에서 확인하거나 threshold를 fitting하는 과정이다. full-input model에 대해 calibration했다고 partial-input model에서도 같은 성질이 유지된다고 볼 수 없다. mask가 바뀌면 predictor의 error 구조가 달라지고, policy가 어려운 episode에서 다른 mask를 선택하면 calibration population도 달라진다.

교육용 binary 예시를 보자. calibration에서 mask A와 B가 같은 수로 관측되었고 confidence score가 두 mask에서 0.9라고 하자. 실제 error는 A에서 0, B에서 0.2다. pooled error는 0.1이어서 score 0.9와 맞아 보인다. 그러나 deployment policy가 B만 만드는 population에서는 error가 0.2다. 가중치와 score를 바꾸지 않았는데도 pooled calibration의 해석이 달라진다.

이 계산은 adaptive policy가 반드시 calibration을 깨뜨린다는 증명이 아니다. frozen policy를 calibration과 final의 같은 분포, 독립 episode에 동일하게 실행한다면 원본 episode를 terminal history로 변환하는 과정 자체를 calibration pipeline에 포함할 수 있다. 필요한 것은 어떤 population·policy·mask 생성 규칙에 대해 결과가 유효한지 명확히 하는 일이다.

특히 confidence threshold가 action stopping에도 영향을 주면 threshold를 바꾸는 순간 관측 trajectory가 바뀔 수 있다. post-acquisition gate만 바꾸는 것과 confidence가 올라올 때 acquisition을 멈추는 것은 다르다. 전자는 같은 terminal packet에서 diagnose/abstain을 바꾸지만 후자는 packet 자체의 분포를 바꾼다. 이 구별 없이 calibration threshold를 사후 조정하지 않는다.

첫 policy-only 실험은 common predictor score·loss·penalty와 공통 gate convention을 고정한다. 그래야 정책이 정보 획득과 stopping을 어떻게 했는지 비교할 수 있다. 각 policy마다 다른 threshold를 fitting해서 목표 risk를 맞추는 실험은 전체 calibrated-system 비교로 따로 보고할 수 있다. 그때 변화한 coverage가 acquisition 때문인지 calibration 때문인지 설명해야 한다.

calibration set은 dev와 분리한다. dev에서 policy·mask mixture·hyperparameter를 선택하고 calibration에서 보정 artifact를 만든 뒤 final에서 고정한다. calibration error를 보고 policy를 다시 선택하면 selection을 반영할 새로운 calibration 설계가 필요하다. final label을 보고 threshold를 바꾸는 것은 허용하지 않는다.

Conformal prediction beyond exchangeability (https://arxiv.org/abs/2202.13415v5)는 exchangeability가 깨질 때의 가정과 방법을 명시한다. 이름에 conformal이 들어간 도구를 썼다고 arbitrary shift에 대한 보장이 생기는 것이 아니다. same-rig dependent cycles, condition combinations의 분포 이동, policy-dependent masks와 stopping은 실제 가정 검토가 필요한 요소다.

conformal prediction set의 label coverage와 여기의 selective diagnosis coverage도 다른 양이다. prediction set coverage는 정답이 반환한 집합에 포함되는 확률이고, diagnosis coverage는 전체 eligible episode 중 시스템이 답한 비율이다. 같은 영어 단어를 쓰지만 분모와 사건이 다르다. 둘을 함께 평가하면 각 정의를 따로 표시해야 한다.

mask별로 calibration할 때는 작은 support 문제가 생긴다. action M개면 mask 조합은 매우 많고 모든 mask에 충분한 독립 group을 갖추기 어렵다. mask를 사후 합쳐 threshold를 좋게 만들지 않으며, 충분한 support가 없는 stratum은 uncertainty가 크거나 blocked라는 상태로 남긴다. 많은 반복 seed가 부족한 독립 calibration group을 대신하지 않는다.

또한 episode의 모든 단계에서 score를 여러 번 보고 처음 기준을 넘은 시점에 멈추면 optional stopping 구조가 된다. 고정 한 시점의 평가와 같은 보장이라고 단정하지 않는다. 첫 주 문서는 risk/coverage를 실측 정의로 제시할 뿐 현재 자료에서 통계적 guarantee를 인증하지 않는다. finite Bellman certificate는 별도의 model-optimality 문장이며 calibration 문장으로 대체되지 않는다.

15. Hydraulic 분할·미래 정보·누락 채널의 실제 제약

현재 원시 준비가 가장 직접 대응하는 과제는 Hydraulic의 네 component head다. 공식 metadata가 설명하는 60초 주기의 PS1은 100 Hz, FS1은 10 Hz, TS1은 1 Hz이며 각각 6,000·600·60개 값으로 확인되었다. source-reported unit과 실제 파일 shape를 구분한다. 실제 시간 측정 trace를 따로 확보하지 않고 metadata의 표본율을 실측했다고 말하지 않는다.

profile의 네 target과 다섯 번째 안정성 flag를 분리한다. flag 0은 안정 조건, 1은 static condition에 아직 도달하지 않았을 수 있다는 감사 정보다. 이를 label이 맞지 않은 데이터로 자동 제거하는 규칙으로 쓰지 않는다. filter를 선택하면 평가 전에 목적·규칙을 고정하고 제외 수를 공개해야 한다.

실제 audit의 flag count는 0이 1,449개, 1이 756개다. 같은 네 target이 인접하게 유지된 구간은 194개다. 이 구간은 profile에서 기계적으로 계산한 연속 run이다. 같은 조건을 재설정했거나 이전 온도·부하가 다음 구간으로 이어졌을 수 있으며, 서로 독립인 실험 블록의 출처를 보여주지 않는다.

따라서 독립-block 평가의 train/dev/calibration/final은 현재 blocked다. 임의로 194 run에 group ID를 붙이고 independent count라고 부르지 않는다. 필요한 자료는 실험 batch, 반복 운전·reset, 시험 날짜·장치 이력 등 어떤 단위가 의존성을 공유하는지 확인할 provenance다. 독립성을 증명할 자료가 없으면 unknown으로 유지한다.

within-rig chronological sensitivity는 다른 선택이다. 과거 구간에서 train하고 뒤 구간에서 평가하며 중간 gap과 history 전달을 선언할 수 있다. 이 방법은 random shuffle보다 정보 흐름을 더 명확하게 만들 수 있지만 같은 장치의 시간 의존성이 사라지는 것은 아니다. 실제 성능을 보고해도 independent asset generalization으로 해석하지 않는다.

leave-combination-out은 알려진 개별 class 수준의 새 조합을 평가한다. 예를 들어 cooler 20과 valve 80을 각각 train에서 봤지만 그 조합을 final에서 처음 볼 수 있다. 각 head 수준의 지지 수를 확인해야 하고, 한 class 자체를 train에서 제거하면 unknown-class 분류 문제로 바뀐다. 새 조합과 새 장비 transfer를 같은 이름으로 부르지 않는다.

primary time setting은 완료된 기록 주기다. 한 주기가 끝난 후 정해진 source window를 조회하는 archived evidence problem으로 시작한다. 이 경우 주기 끝까지의 값을 쓰는 것은 정의상 미래 누출이 아니다. 실제 sensing을 중간에 켜거나 끄는 제어 실험도 아니다.

online-prefix variant는 별도다. 주기의 20초 시점에서 의사결정한다면 100 Hz pressure 창의 stop은 최대 2,000이어야 한다. [0,6000)을 가져오면 미래 40초를 사용한 것이다. 온도와 유량도 같은 decision time에 맞춰야 하며 window의 시간 대응을 검사한다. prefix에서 annotation이 같은 상태를 대표하는지, 불안정 주기의 정답 해석이 무엇인지도 재검토한다.

CE와 CP는 공식 설명에 virtual로 표시되어 있고 SE의 실제 계산 dependency는 확인되지 않았다. 세 값을 모두 초기 evidence에서 제외한다. virtual channel을 허용하려면 실제 raw prerequisites와 transform version을 확인하고 해당 읽기·계산을 청구해야 한다. target과 강하게 상관된 derived quantity를 무료로 주면 source acquisition 연구의 정보 환경이 달라진다.

현재 준비되지 않은 11개 physical channel은 자연적인 측정 고장이 아니라 이번 제한 추출의 결과다. full14 protocol에서 일부를 의도적으로 숨기는 simulated availability mask와 이 partial intake를 혼동하지 않는다. v3 authored no_pressure/no_flow mask도 실제 설비 센서 고장이 아니다. 각각의 missing stratum과 denominator를 구분한다.

원본 class code, cycle index, condition-bearing name을 policy-visible ID로 쓰면 시간 순서나 이름으로 label을 추측할 수 있다. evaluator는 원본 row mapping을 유지하되 policy에는 opaque ID를 준다. 실제 chronological task에서 public time context를 허용하려면 모든 정책에 같은 권한을 주고, time-only baseline으로 shortcut 가능성을 별도 점검한다.

UCI의 raw-data license는 CC BY 4.0이며 dataset citation과 원본 링크를 보존한다. library나 다른 연구자의 code license를 이 license로 덮어쓰지 않는다. 현재는 새 의존성·외부 모델·추가 raw download 없이 기존 확보와 schema receipt를 설계 근거로 사용한다.

16. 무엇을 바꾸고 무엇을 고정하는 실험인가

주 실험의 intervention은 acquisition/stopping policy다. predictor φ, preprocessing, numerical module bank, initial packet, source 권한, availability, marginal cost function과 평가 정답은 고정한다. 이 조건에서 terminal loss·coverage·cost가 달라지면 policy trajectory와 stop 선택의 효과라고 해석할 수 있다. 아직 industrial policy trial은 없으며 아래는 실행 계약이다.

대조군은 initial-only, expert order, random feasible, actual-cost greedy terminal gain, pair lookahead부터 둔다. initial-only는 추가 evidence 없이 시작 packet으로 종료한다. expert order는 사전에 정한 분석 순서를 따르며 예산·불가 action을 처리하는 규칙을 고정한다. random feasible은 기록한 seed와 동일 episode에서 가능 action 중 선택한다. greedy와 lookahead는 같은 estimator와 terminal gate를 써 horizon 차이를 분리한다.

강한 learned AFA도 필요하다. L2M, NM-PPG, BRiG 기반 adapter의 native setting과 변경점을 먼저 감사한다. 논문에 method 이름을 썼다고 그 알고리즘을 재현한 것은 아니다. 다른 predictor를 쓰는 native system은 별도 complete-system 비교이고, fixed predictor에 policy만 이식한 버전은 adaptation이다. 외부 원문의 성능 수치와 우리 future trial을 같은 표에 합치지 않는다.

산업 선행연구도 포함한다. Bian 등의 resource-aware acquisition (https://www.mdpi.com/1424-8220/26/16/5065)은 low-cost screening과 추가 confirmation을 다루므로 산업 sensing의 최초 사용을 주장할 수 없다. 이번에는 공식 abstract와 서지를 확인했으며 전체 HTML과 code의 cache/disclosure 동작을 감사하지 못했다. 그 연구가 shared dependency를 다루지 않았다고 단정하거나 원문 결과를 우리의 성능표에 복사하지 않는다.

charge와 disclosure의 2×2는 주 성능 실험과 다른 mechanism diagnostic이다. 첫 요인은 naive duplicate charging 또는 deduplicated charging이다. 둘째 요인은 requested-only correct disclosure 또는 free-derived leak이다. estimator와 predictor는 고정해 evaluator가 회계 오류·정보 오류에 얼마나 민감한지 확인한다.

charge	correct disclosure	free-derived leak
naive duplicate	중복 비용 진단	비용·정보 경계 모두 잘못된 진단
deduplicated	유효 인터페이스 기준	계산된 prerequisite를 공짜로 공개하는 누출 진단

leak cell은 의도적으로 유효하지 않은 처리다. 그 cell에서 정확도가 높아져도 새로운 policy의 좋은 결과로 보고하지 않는다. 숫자와 mask가 왜 바뀌었는지, 실제 공개되지 말아야 할 prerequisite를 넣었는지 검사하는 boundary experiment다. valid 산업 성능표에는 dedup+correct로 실행한 정책들만 넣는다.

naive charge도 모든 primary competitor에 적용할 손해 장치가 아니다. 실제 공유 ledger가 있는 문제에서는 greedy와 expert도 같은 discount를 받아야 한다. 새 방법은 dedup, baseline은 naive로 청구하면 정책 능력이 아니라 비용 정의를 바꾼 비교가 된다. 9 vs 15를 objective improvement처럼 보고하지 않는 이유다.

state ablation은 executor의 실제 비용을 바꾸지 않는다. policy input에서 ledger representation을 뺄 수 있지만 quote API와 full history로 그 정보를 다시 추론할 수 있는지 명시한다. 이미 equivalent quote를 주면서 "cache 정보를 제거했다"고 이름만 붙이면 ablation의 의미가 없다. 반대로 baseline에 정확한 action legality도 알려주지 않는다면 권한 차이가 섞인다.

비교할 수 있는 좁은 방법은 같은 quote·합법 action을 주되 learned state representation에 explicit resource features를 포함했는지 비교하는 것이다. 이는 representation과 학습 효율의 차이를 검사한다. cache를 reconstruct할 수 있는 성공-only control에서는 정보적 이득을 기대하지 않고, value-independent failure 또는 사전 정의한 warm-history stratum에서 필요한 state 차이를 검사한다.

horizon ablation은 같은 predictor와 outcome estimator에서 한 단계와 두 단계 이상을 비교한다. 더 깊은 planner가 더 많은 training data나 더 큰 model을 쓰면 depth만의 효과가 아니다. 정확한 pair enumeration이 더 강한 learned method와 비슷한 결과를 낼 수도 있다. 이 경우 복잡한 정책의 필요성이 약해진다는 결과를 인정한다.

별도 source identity control은 output 값은 유지하되 원래 공유하던 입력을 action별 독립 key로 취급하는 counterfactual interface다. 값의 통계적 독립성을 만드는 것이 아니다. pressure_mean과 pressure_delta의 상관 구조는 그대로일 수 있다. "independent-source control"이라는 이름을 쓸 때 이 차이를 명확히 한다.

같은 단위 예산 B에서 control의 비용이 달라지면 feasible trajectory도 바뀐다. 이를 숨기지 않고 actual-budget curve와 각 interface의 maximum-cost fraction sensitivity를 구분해 보고할 수 있다. 조건별 policy gain 차이는 공유 비용 해석을 도울 수 있지만, 모든 환경 차이를 제거한 순수 인과 효과라고 과장하지 않는다.

특히 source sharing을 없애도 XOR 보완성은 남을 수 있다. non-myopic gain이 control에서도 유지되면 "공유 비용 때문에 이겼다"는 설명이 약해질 수 있지만 보완 정보 가설 자체가 자동 반증되는 것은 아니다. 서로 다른 mechanism을 합친 한 주장보다 어떤 stratum에서 어떤 차이를 예상하는지 분리하는 편이 깊은 검증이다.

실제 resource 비교에서는 cold/warm 논리 cache, OS warm 상태, missing/failure stratum과 planning cost를 함께 기록한다. seed를 늘려 시행 순서를 바꾸고 paired episode를 유지하는 설계는 timing noise를 볼 수 있지만 scientific group 수를 늘리지는 않는다. total-system 효율을 주장하려면 policy planning과 optional LLM 호출 비용까지 포함해야 한다.

17. 정량 지표의 계산 예와 반증 기준

기본 분모는 eligible unique episode다. N개 중 D개를 diagnose하고 A개를 abstain했다면 N=D+A이다. failed 실행을 별도 episode로 제외하는지, 실패 후 abstain으로 종료하는지 사전에 정해야 한다. 현재 제안에서는 실행 failure와 그 비용을 보존하고 terminal 처리 규칙을 고정한다. 계산이 어려웠다는 이유로 낮은 성능 episode를 사후 삭제하지 않는다.

coverage=D/N selective vector risk=(diagnosed 중 하나 이상 head를 틀린 수)/D diagnosed component error=(diagnosed의 전체 head mistake 수)/(4D) terminal loss=(diagnosed의 component loss 합+λA·A)/N

교육용 점검 예시로 eligible episode 100개, diagnose 60개, abstain 40개를 두자. diagnose 중 12개에서 적어도 한 head를 틀렸고, 전체 head mistake는 18개라고 하자. coverage는 60/100=0.6, selective vector risk는 12/60=0.2, diagnosed component error는 18/(4×60)=0.075다.

diagnosed component loss의 합은 18/4=4.5다. λA=0.3이면 abstention loss는 40×0.3=12이고 terminal loss는 (4.5+12)/100=0.165다. 같은 결과를 "선택적 오차가 7.5%"라고만 표현하면 vector risk 20%와 coverage 60%가 사라진다. 정확히 어떤 error를 말하는지 구분해야 한다.

이 예시의 λA를 0.1로 재계산하면 같은 고정 terminal 기록에서 (4.5+4)/100=0.085가 된다. λA=0.5이면 0.245다. 이는 loss 정의의 민감도이며 새로운 policy를 학습한 결과가 아니다. 실행 전 primary λA를 고정하고 사전에 선언한 sensitivity 값만 보고한다. 최종 결과가 좋아 보이는 λA만 선택하지 않는다.

만약 diagnose가 0이면 coverage=0, terminal loss=λA다. selective vector risk와 diagnosed component error는 D=0이라 NA이다. all-abstain이 정확한 시스템이라는 요약은 허용되지 않는다. per-head probability의 평균 confidence도 그 빈 denominator를 해결하지 못한다.

비용은 모든 eligible attempt에서 합산한다. 예를 들어 총 800개의 declared evidence unit을 사용했다면 평균은 800/100=8이다. diagnose 60개로만 나누면 abstain과 failure에서 쓴 비용이 빠진다. planning 총 시간이 10초였다면 평균은 0.1초다. 둘은 서로 다른 단위이고 normalized weights가 정해지기 전에는 더해 하나의 비용처럼 쓰지 않는다.

주 budget별 policy 결과는 같은 cycle·초기 packet·availability에서 paired comparison한다. 예를 들어 policy A와 B의 terminal loss 차이를 episode별로 계산한 뒤, 독립 group이 확인되면 group 단위의 평균·불확실성을 분석할 수 있다. mask와 seed 반복은 같은 episode 안의 nested condition으로 유지한다. 100 episode를 10번 반복해 1,000개의 독립 표본을 만들었다고 표현하지 않는다.

risk-coverage curve는 공통 score의 threshold를 바꾸어 어떤 episode를 답할지 시각화하는 기술적 결과다. score가 mean component risk이면 curve의 risk도 component인지 vector인지 명시한다. sparse mask에 sample이 없거나 diagnose가 0인 끝점은 NA로 둔다. curve의 면적이나 극단점 하나가 statistical calibration guarantee를 대신하지 않는다.

최종 hypothesis에는 primary comparator와 budget, useful margin을 정해야 한다. practical margin이 없으면 아주 작은 numerical 차이를 근거로 의미 있는 개선이라고 부를 수 있다. 현재 이 값들은 unlocked다. 독립 block, predictor와 resource protocol이 준비된 뒤 dev 단계에서 정하고 final 전 고정한다.

정책 우위 해석을 반증하는 핵심 경우는 다음과 같다. 가장 강한 matched comparator가 같은 또는 더 낮은 terminal loss를 보인다. predictor·공개 권한·actual marginal cost·planning cost를 맞추면

gain이 없어진다. baseline에 source를 중복 청구하거나 새 정책에 prerequisite를 무료 공개한 경우만 gain이 있다. 이때 positive fixture score나 높은 confidence로 주장을 구제하지 않는다.

mechanism 설명은 더 좁게 검사한다. failure-cache stratum에서 state 표현 차이가 없고 exact quote/history가 이미 충분했다면 explicit ledger 추가의 필요성이 약해진다. complementary stratum에서 pair lookahead가 충분하면 더 복잡한 learned planner의 필요성이 약해진다. separate source identity control에서 cost-specific gain이 줄지 않으면 sharing이 주요 원인이라는 설명은 재검토한다.

경계 위반은 성능과 별도로 run을 무효화한다. hidden label, stable flag, virtual shortcut, 미래 샘플, 다른 episode 캐시, 잘못된 source window, 초과 예산 허용, failure 비용 누락, fabricated industrial completion은 해당 실행의 해석을 막는다. 이때 결과를 성공 평균에 넣지 않으며 원인과 원본 기록을 보존한다.

다만 해시 검증 통과가 독립 scientific design의 인증은 아니다. 입력과 output의 로컬 정합성, code/runtime pin, semantic replay를 확인해도 원시 annotation의 인과적 의미나 새로운 설비 전이까지 증명하지는 않는다. 소프트웨어 품질과 과학적 반증 가능성을 둘 다 필요로 하지만 서로 대체하지 않는다.

18. 네 페이지 원고의 읽기 지도와 예상 Q&A

원고는 첫 주 설계 문서이므로 결과 논문처럼 문장을 압축하면 안 되는 부분이 있다. 예시가 어떤 유형인지, actual partial intake가 어디까지인지, guarantee의 가정이 무엇인지 유지한다. 상세 개념은 이 해설서에 두되 4페이지 안에는 문제·formal counterexample·평가 대비·반증 경계를 남긴다.

원고 의미 페이지	중심 내용	상세 설명서 연결
1	질문·fixed predictor·기존 연구와 좁은 기여	1–4절
2	typed ledger·failure-cache·XOR	5–10절
3	실제 partial readiness·pipeline·공정한 실험	12–17절
4	finite Bellman·lower bound·반증	11, 17절

Q1. 그냥 feature acquisition에 이름만 붙인 것 아닌가? sequential acquisition과 non-myopic planning 자체는 기존 연구다. 우리는 typed analysis의 shared source, internal computation과 실제 disclosure를 분리했을 때 공정한 비용·상태 비교가 어떻게 달라지는지 질문한다. 이 설계가 실제 산업에서 추가 가치가 있는지는 strong adapted baseline과 mechanism control로 검증해야 한다.

Q2. confidence가 좋아지는 것만 보면 되지 않나? confidence score의 변화는 fixed predictor의 output이다. 아직 보지 않은 분석의 outcome 분포와 미래 비용은 별도다. 같은 confidence여도 cache 때문에 다음 action이 가능하거나 불가능할 수 있다. 실제 terminal loss와 cost를 보지 않고 confidence 상승만으로 정책 우위를 채점하지 않는다.

Q3. 캐시가 있으면 무료 입력을 준 것 아닌가? server가 raw를 보유하는 것과 policy가 그 값을 본 것은 다르다. exact identity를 재사용하면 acquisition charge를 할인할 수 있지만 undisclosed output을 predictor에 넣을 수 없다. initial/prewarm 비용의 처리와 반환 overhead도 기록해야 한다.

Q4. 값과 mask만으로도 cache를 알 수 있지 않나? 성공-only deterministic DAG와 같은 빈 초기 cache, 완전한 request provenance에서는 그렇다. explicit O/C가 추가 정보가 아닐 수 있다. 실패가 raw를 남기고 수치는 반환하지 않는 조건부 사례에서는 같은 mask가 다른 cache를 숨길 수 있다. 이 차이를 전제로 state claim을 제한한다.

Q5. XOR 반례를 넣으면 새 알고리즘이 증명된 것인가? 아니다. XOR는 one-step gain에 diminishing returns가 없을 수 있음을 직접 계산한 예다. pair lookahead나 exploration도 해결할 수 있고 기존 non-myopic 방법이 이미 이 방향을 다룬다. 반례는 greedy와 guarantee의 적용 조건을 설명하며 산업 improvement를 인증하지 않는다.

Q6. exact planner가 industrial optimum을 보여주나? 알려진 유한 joint model과 finite action/outcome, 올바른 Markov state·charge라는 가정에서만 reference optimum이다. 산업 classifier probabilities만으로는 next-outcome kernel을 얻지 못한다. 연속 큰 상태에서 계산·모델 오류·shift에 대한 보장은 없다.

Q7. all-input teacher가 upper bound 아니냐? 하나의 learned teacher는 partial predictor보다 실제로 나쁠 수 있다. free-information Bayes relaxation은 추가 정보를 무시할 자유와 정확한 prior를 가진 다른 객체다. 참고 teacher line과 mathematical lower/upper bound를 분리한다.

Q8. 2,205 cycles면 calibration에 충분하지 않나? 숫자만으로 판단할 수 없다. 같은 장치와 운전 이력의 의존성을 확인해야 하며 독립 block은 unknown이다. 194 adjacent condition run도 독립 batch 인증이 아니다. class·mask·failure stratum별 지지와 split provenance를 확인해야 한다.

Q9. 실제 기술 도구를 무엇으로 만들 계획인가? local TSV/profile reader와 hash verifier, typed registry, read-only evidence executor, numerical masked predictor, acquisition selector, calibration runner, protected evaluator, semantic verifier를 단계별로 만든다. 현재 kernel과 partial intake가 있고 산업 predictor·policy·calibration은 미구현 또는 미학습이다.

Q10. LLM은 어디에 쓰나? 선택적으로 다음 legal analysis 또는 stop을 제안하는 policy client다. 숫자 예측과 confidence calibration은 frozen 수치 도구가 담당한다. hidden raw나 label을 LLM에 주지 않으며 실제 호출하려면 model·전송 범위·token/cost/time 한도를 먼저 고정한다. 현재 호출은 0이다.

Q11. 일부러 leak를 허용한 2×2는 위험한 비교 아닌가? 의도적으로 invalid cell을 만들어 harness가 정보·비용 오류를 탐지하고 결과가 얼마나 바뀌는지 확인하는 진단이다. valid 방법의 성능으로 보고하거나 산업 성능표에 합치지 않는다. primary comparator들은 모두 같은 dedup+correct executor를 사용한다.

Q12. 어떤 결과가 나오면 이 연구가 실패했다고 인정하나? 강한 matched baseline과 equivalent하거나 total planning cost를 포함하면 이득이 없을 수 있다. gain이 predictor 차이·누출·baseline double charging에서만 나온 경우 policy claim을 거절한다. 결과가 없거나 입력·독립 group이 불충분한 단계에서는 NR/blocked를 유지한다.

첫 주의 구체적 산출물은 4개 의미 페이지 원고, 장문 해설, 방법 계약, source/claim inventory와 별도의 계산 검토다. 이는 industrial pre-registration이 완성되었다는 상태가 아니다. 다음 실행의 우선순위는 독립 group과 full14/restricted3 선택, budget·λA·resource cap의 고정, 작은 실제 predictor와 mask/outcome protocol, strong adapter와 별도 calibration이다.

지금 보고할 수 있는 것은 trusted reference mechanics와 실제 limited Hydraulic schema preparation, 그리고 가정을 드러내는 계산 예시다. 아직 없는 산업 결과를 채워 넣지 않는다. 설계의 깊이는 큰 알고리즘 이름을 붙이는 데서 생기지 않고, 같은 정보와 자원에서 어떤 선택이 어떤 원인으로 달라지는지를 계산·기록·반증할 수 있게 만드는 데 있다.